

while Loop in Java

The **while loop** is a control flow statement that repeatedly executes a block of code as long as a specified condition evaluates to **true**. It is used when the number of iterations is not known in advance and depends on the condition being checked.

Syntax:

```
while (condition) {  
    // Code to be executed as long as condition is true  
}
```

- **condition**: A boolean expression that is evaluated before each iteration.
- **Code block**: The code inside the **{}** braces will execute as long as the condition is **true**.

The condition is evaluated **before** each iteration, meaning that if the condition is **false** initially, the code inside the loop will not execute even once.

How It Works:

1. The **condition** is evaluated. If it is **true**, the code inside the loop is executed.
 2. After executing the code, the condition is checked again.
 3. This process continues until the condition evaluates to **false**.
 4. If the condition is **false** from the beginning, the loop is skipped entirely.
-

Example 1: Basic **while** Loop

```
public class Main {  
    public static void main(String[] args) {  
        int count = 1;  
  
        while (count <= 5) {  
            System.out.println("Count: " + count);  
            count++; // Increment count by 1  
        }  
    }  
}
```

Explanation:

- The loop starts with `count = 1`.
- The condition `count <= 5` is `true` initially, so the code inside the loop is executed.
- After each iteration, the value of `count` is incremented by 1.
- This process continues until `count` becomes 6, at which point the condition `count <= 5` becomes `false`, and the loop terminates.

Output:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Example 2: Infinite `while` Loop

```
public class Main {
    public static void main(String[] args) {
        while (true) {
            System.out.println("This is an infinite loop");
        }
    }
}
```

Explanation:

- The condition is always `true`, making the loop run indefinitely.
 - This is an **infinite loop**, which will keep executing the code inside the loop without stopping unless there is a break condition or an external interruption.
 - **Note:** Infinite loops should be used with caution, as they can cause the program to hang.
-

Example 3: `while` Loop with Break Statement

```
public class Main {
    public static void main(String[] args) {
        int number = 1;
```

```

        while (number <= 5) {
            if (number == 3) {
                break; // Exit the loop when number equals 3
            }
            System.out.println("Number: " + number);
            number++;
        }
    }
}

```

Explanation:

- The loop will run until the `number` reaches 3.
- When `number == 3`, the `break` statement is executed, which immediately exits the loop.
- The program will print numbers from 1 to 2, and the loop will stop before printing 3.

Output:

```

Number: 1
Number: 2

```

Example 4: `while` Loop with Continue Statement

```

public class Main {
    public static void main(String[] args) {
        int number = 0;

        while (number < 5) {
            number++;
            if (number == 3) {
                continue; // Skip the rest of the code for this iteration
                when number is 3
            }
            System.out.println("Number: " + number);
        }
    }
}

```

Explanation:

- The `continue` statement causes the loop to skip the current iteration when `number == 3`.
- As a result, the number 3 is not printed, and the loop continues with the next iteration.

Output:

```
Number: 1  
Number: 2  
Number: 4  
Number: 5
```

Key Points:

1. **Condition Checking:** The condition is checked before each iteration. If the condition is `false` initially, the loop will not execute.
 2. **Infinite Loop:** If the condition always evaluates to `true`, the loop will run infinitely unless a `break` statement is used or the loop is interrupted externally.
 3. **Increment/Decrement:** Make sure to update the variable(s) involved in the condition (like incrementing or decrementing) to avoid an infinite loop.
 4. **Control Statements:** You can use `break` to exit the loop early or `continue` to skip the current iteration and move to the next one.
-